Parallel and Distributed Systems: Paradigms and Models
Project 1 (MergeSort) Report

Davide Marchi (602476)
d.marchi5@studenti.unipi.it

August 2025

Contents

1 Introduction

I present a complete study of a distributed out-of-core MergeSort. The study proceeds in four steps. First, I describe the problem and the data layout. Second, I implement four solutions that share a common I/O pipeline while differing in how they exploit parallelism: a sequential baseline, an OpenMP task based version, a FastFlow farm based version, and a multi node hybrid MPI solution that reuses the single node core. Third, I evaluate these versions under controlled workloads and hardware configurations, measuring runtime, speedup, and efficiency. Finally, I discuss the results and propose an approximate cost model for the distributed solution.

All implementations work on files that are potentially larger than available RAM. Each program first scans the unsorted file to build an in memory index of keys and positions and lengths, then sorts this index, then rewrites a new file in key order by fetching payloads at the recorded positions. To keep memory usage proportional to the number of records, the in memory structure stores only keys, offsets, and lengths. The write phase can be highly variable on a shared cluster, so the primary performance analysis focuses on the reading and sorting stages. The full pipeline remains functionally correct and the final output is validated.

The objectives are:

- demonstrate correctness of all variants by verifying that keys are globally sorted in the *written* output file and that records are preserved;
- quantify single node scalability with strong and weak scaling for OpenMP and FastFlow;
- quantify multi node scalability for the MPI hybrid, using one rank per node and varying threads per rank;
- interpret bottlenecks that arise from memory access, tasking overheads, and communication.

Structure. Section 2 defines the problem and data model. Section 3 discusses memory and I/O considerations. Section 4 presents single node implementations and their evaluation. Section 5 presents the multi node implementation and its evaluation. Section 6 proposes an approximate cost model and discusses bottlenecks and overlap.

2 Problem description

2.1 Data model and constraints

The input is a large binary file that stores an array of records. Each record consists of a sortable key, a length field, and a payload. The payload size can be large compared to the key, and the file may exceed available RAM, which motivates an out-of-core strategy.

I use the following logical layout for each **Record**:

- **key**: 64 bit unsigned integer used for ordering;
- **len**: 32 bit unsigned integer that gives the payload length in bytes, with len $\leq P$ for a dataset parameter P;
- **payload**: a byte array of length len.

Sorting records in place is impractical when the file is larger than RAM. Instead, I construct an auxiliary in memory index. Each entry is an **IndexRec** that stores only what is needed to order and later locate the record:

- **key**: 64 bit unsigned integer, copied from the record;
- **pos**: byte offset in the unsorted file where the record begins;
- **len**: 32 bit unsigned integer, copied from the record, used to stream exactly len bytes during rewrite.

The index has length N, one entry per record. Its memory footprint is $O(N)$ and is independent of the maximum payload size P. This design keeps memory predictable and minimizes disk traffic during random access.

Correctness means that the output file contains exactly the input records in nondecreasing key order. I validate this by scanning the *written* output file from disk, checking that keys are ordered and that the record count matches the input.

2.2 Pipeline, distributed setting, and metrics

The pipeline is the same across all variants:

1. **Index build.** Scan the unsorted file, read `key` and `len` per record, and fill the `IndexRec` array with {key, pos, len}. File access uses memory mapping to support efficient random access patterns and to avoid large user space buffers.
2. **Index sort.** Sort the `IndexRec` array by key. The sequential baseline uses `std::sort`. The parallel versions use a task based mergesort with a base case threshold that delegates small subarrays to `std::sort`.
3. **Rewrite.** Create the output file, then for each entry in sorted order fetch the record at `pos` from the unsorted file and write it to the next position in the output file, streaming exactly `len` bytes for the payload. This preserves the original payloads without storing them all in memory.
4. **Validation.** Scan the output file to confirm nondecreasing key order and that the record count is unchanged.

In the multi node variant, I run one MPI rank per node. A designated rank reads the file and builds the global index, then partitions that index into contiguous slices and sends one slice to each rank. Each rank sorts its slice locally. The globally sorted order is

obtained through a sequence of pairwise merges across ranks. The final rank rewrites the output file and performs validation. Communication is limited to index data and merged index segments, which keeps memory usage predictable and decoupled from payload size.

The primary metrics are total time for index build plus sort, speedup $S_p = T_1/T_p$, and efficiency $E_p = S_p/p$. For single node studies, p is the thread count. For distributed studies, p is the product of nodes and threads if noted, or the number of threads per rank when speedup is plotted against nodes. Strong scaling holds N fixed while varying p. Weak scaling grows N proportionally to p.

3 Memory and I/O considerations

My design keeps memory usage proportional to the number of records rather than to the maximum payload size or to an arbitrary buffer. I maintain an in memory index of length N, where each entry stores only the information required to order and later locate a record: $\{\texttt{key}, \texttt{pos}, \texttt{len}\}$. The footprint is therefore $O(N)$ with a small constant factor that depends on the sizes of these fields, and it does not depend on the maximum payload parameter P. During rewrite I stream each payload directly from the unsorted file into the output without caching all payloads in RAM.

3.1 Why I use memory mapping

I use `mmap` for both reading and writing. This choice avoids explicit user space I/O buffers and lets the operating system handle paging and readahead. It is well suited to the two access patterns in this project:

- During index build, I scan the file record by record and read only `key` and `len`. Mapping the file allows the kernel to fault in pages efficiently and to reuse them across passes when the page cache is warm.
- During rewrite, I must fetch payloads at the original positions and emit them in sorted order. This requires many random reads of the source and a sequential write to the destination. With `mmap` I can copy exactly `len` bytes per record from the mapped source region to the mapped destination region without building large staging buffers.

This approach keeps per record work bounded and predictable, and it aligns with the constraint that the file may be much larger than available RAM.

3.2 Single threaded disk access

On the shared cluster, attempts to parallelize disk access did not produce stable gains. Running multiple threads that read or write concurrently tended to increase queue contention and amplify interference from other users on the same storage. I therefore adopt a conservative policy:

1. I perform file I/O with a single thread at a time.
2. I exclude the rewrite phase from the primary performance graphs and focus the analysis on index build plus sorting.

This policy isolates the compute and indexing behavior of the implementations from storage noise and makes cross run comparisons meaningful.

3.3 Variability of the write phase

The write phase is highly volatile on the target system. In three executions of the same configuration, the time to rewrite the sorted file differed by almost an order of magnitude. Table 1 reports the raw measurements.

Table 1: Observed variability of the rewrite phase for identical runs

Run	Write time (s)	Notes
1	68.97	light system load
2	264.69	moderate contention
3	503.60	heavy contention

Given this spread, including rewrite time in the main figures would mask the effects I want to study. I still execute the full pipeline in every run and I always validate correctness on the *written* output file from disk, but I report index build plus sorting as the primary metric in Sections 4 and 5.